

Bus Management System — Complete Software Documentation

Project Name: Crowd Sense Hub

Version: 1.0.0

Platform: iOS, Android, Web

Date: May 2026

Table of Contents

1. Project Overview
 2. Technology Stack
 3. System Architecture
 4. Frontend Architecture
 5. Backend and Services
 6. Database Schema
 7. Screens and Features
 8. Reusable Components
 9. Navigation and Routing
 10. State Management and Data Flow
 11. Authentication and Security
 12. APIs and Third-Party Integrations
 13. IoT Hardware Integration
 14. Project Structure
 15. Conclusion
-

1. Project Overview

Crowd Sense Hub is a comprehensive real-time Bus Management System developed as a cross-platform mobile application using React Native and Expo. The system serves three distinct user roles: Students (passengers), Drivers (bus operators), and Administrators. It provides a complete ecosystem for modern public transit management including digital ticketing, real-time GPS tracking, automated passenger counting, QR-based boarding verification, and live administrative dashboards.

The system integrates IoT hardware through an ESP32 microcontroller equipped with GPS modules, infrared sensors for passenger detection, an OLED display for onboard information, and a buzzer for audio feedback. All data is synchronized in real time through Google Firebase Realtime Database.

Core Objectives

The system addresses the lack of real-time visibility into bus occupancy, location, and schedule adherence in college and public transit systems. It provides a mobile application combined with IoT hardware to track passengers, monitor GPS location, and deliver live dashboards for administrative oversight.

User Roles

The system defines three distinct user roles with tailored interfaces and permissions. Students access the passenger application to search routes, book tickets, view live bus locations, scan QR codes for boarding, and trigger SOS alerts in emergencies. Drivers use the operator panel to scan passenger QR tickets, monitor passenger counts, and manage boarding operations. Administrators access a real-time monitoring dashboard for route creation and management, viewing analytics, and monitoring system logs.

2. Technology Stack

Frontend Technologies

The frontend is built on React Native version 0.85.3, providing cross-platform compatibility for iOS and Android devices. Expo version 56.0.4 serves as the managed workflow and build toolchain, simplifying development and deployment processes.

Navigation is handled through two React Navigation libraries. React Navigation Native Stack version 7.16.0 manages screen-based transitions, while React Navigation Bottom Tabs version 7.16.2 implements the tab bar navigation for both student and admin interfaces.

Visual design is supported by several libraries. The @expo-google-fonts/poppins library version 0.4.1 provides the Poppins font family. Lucide React Native version 1.16.0 supplies the icon set. Expo Linear Gradient version 56.0.4 enables gradient backgrounds and UI effects.

Location services use Expo Location version 56.0.13 for GPS positioning. Camera functionality uses Expo Camera version 56.0.7 for QR code scanning. Notifications are managed through Expo Notifications version 56.0.13.

Graphics and mapping components include React Native SVG version 15.15.4 for route visualization, React Native WebView version 13.16.1 for embedded Leaflet maps, React Native Safe Area Context version 5.7.0 for device safe area handling, and React Native Screens version 4.25.2 for native screen containers.

Data persistence and utilities use React Native Async Storage version 2.2.0 for local storage and React Native Community DateTimePicker version 9.1.0 for date and time selection.

Backend Technologies

The backend infrastructure relies on Google Firebase. Firebase Realtime Database version 12.13.0 serves as the primary data store with real-time synchronization capabilities. Firebase Authentication version 12.13.0 handles email and password authentication.

An additional backend component is a Node.js Express server version 5.2.1 that functions as an OTP mailer. Nodemailer version 8.0.8 handles email sending through Gmail SMTP. The emailjs-com library version 3.2.0 provides client-side email service integration. CORS version 2.8.6 manages cross-origin request policies.

IoT Hardware Components

The hardware subsystem consists of an ESP32 dual-core microcontroller with integrated WiFi for cloud communication. A GPS Module using the NEO-6M chipset provides real-time bus location tracking. An OLED Display using the SSD1306 driver chip with 128 by 64 pixel resolution shows live passenger count and system

status. Two IR Sensors of the FC-51 type detect passenger entry and exit events. An active buzzer provides audio feedback during boarding and alighting operations.

3. System Architecture

The Bus Management System follows a three-tier architecture comprising the client application, cloud database, and IoT hardware layer.

Client Application Layer

The client application is a React Native mobile app built with the Expo managed workflow. It communicates with Firebase Realtime Database through the Firebase Web SDK version 12.13.0. The application implements state-based navigation where the root component manages authentication state and renders appropriate screen sets based on the user role retrieved from the Firebase database.

Cloud Database Layer

Google Firebase Realtime Database serves as the central data repository. All real-time data including GPS coordinates, passenger counts, ticket records, user profiles, route definitions, booking information, QR scan logs, and SOS alerts are stored and synchronized through Firebase. The database uses JSON-style node structures with real-time listeners that propagate changes to all connected clients within milliseconds.

IoT Hardware Layer

The ESP32 microcontroller operates as the physical data source. It runs firmware that reads GPS coordinates from the NEO-6M module every five seconds and pushes them to the Firebase gps node. Two IR sensors mounted at the bus door detect passenger movement direction through a sequential triggering algorithm. When a passenger enters or exits, the ESP32 updates the passenger count in Firebase and logs the event with a timestamp. The onboard OLED display shows the current passenger count and system status, while the buzzer provides audio confirmation of detected events.

Communication Flow

Data flows in two primary directions. The ESP32 hardware writes GPS data, passenger counts, and event logs to Firebase. The mobile application reads this data through real-time Firebase listeners and displays it to users. When students book tickets or administrators create routes, the mobile application writes data to Firebase, which can then be read by other clients or used for verification during QR scanning.

4. Frontend Architecture

Application Entry Point

The application entry point is the App.js file, which performs several critical initialization tasks. It asynchronously loads the Poppins font family including regular, medium, semibold, bold, and extrabold weights using the useFonts hook from Expo Font. It sets up a Firebase onAuthStateChanged listener to monitor authentication state changes. It reads the user role from the Firebase database path users slash uid slash role to determine whether the authenticated user is a student or an administrator.

Screen Hierarchy

The application uses conditional rendering based on authentication state and user role. When no user is authenticated, the application displays either the WelcomeScreen or the LoginScreen based on a showWelcome state flag. When a student user is authenticated, the application renders a custom floating pill-shaped tab bar with four tabs: Home, Tickets, Stops, and Profile. When an administrator is authenticated, the application renders a standard bottom tab bar with four items: Dashboard, Routes, Driver Mode, and Logout.

Screen Dimensions and Constraints

The application is configured for portrait orientation only. The splash screen uses native resize mode with a background color of E6F4FE. The user interface style is set to light mode. The application supports iOS, Android, and web platforms with tablet support enabled for iOS devices.

Font System

The application uses the Poppins font family from Google Fonts through the Expo Google Fonts package. Five font weights are loaded: regular at weight 400, medium at weight 500, semibold at weight 600, bold at weight 700, and extrabold at weight 800.

5. Backend and Services

Firebase Realtime Database

Firebase Realtime Database is the central data store for the entire application. The Firebase project is identified as bus-management-system-b36c4 with the database URL pointing to the default Realtime Database endpoint. The Firebase Web SDK version 12.13.0 is used for all database operations.

The Firebase configuration includes several parameters. The apiKey provides access to the Firestore REST API. The authDomain specifies the Firebase Authentication domain. The databaseURL points to the Realtime Database endpoint. The projectId identifies the Google Cloud Platform project. The storageBucket specifies the Cloud Storage bucket. The messagingSenderId identifies the Firebase Cloud Messaging sender. The appId uniquely identifies the Firebase application.

Three modules are exported from the Firebase configuration file. The app object is the initialized FirebaseApp instance. The db object is the RealtimeDatabase reference used for all read and write operations. The auth object is the FirebaseAuth instance configured with AsyncStorage persistence for maintaining authentication state across application restarts.

Mailer Server

A standalone Express server handles OTP email delivery. The server runs on Node.js version 18 or higher using Express version 5.x on port 3000. Email delivery is managed through Nodemailer using Gmail SMTP credentials.

The server exposes a single POST endpoint at send-otp. The endpoint accepts a JSON request body containing the recipient email address and the OTP code. Upon receiving a request, the server constructs an HTML email with the Crowd Sense Hub branding, displays the OTP code in a prominent styled box, includes an expiry notice, and sends the message through the Gmail SMTP server.

Utility Modules

The `gpsUtils` module provides mathematical functions for GPS data processing. The `calculateDistance` function implements the Haversine formula to compute the great-circle distance between two geographic coordinates. The `calculateSpeed` function derives speed in kilometers per hour from the distance between two GPS points and the time interval between them. The `smoothSpeed` function applies an exponential moving average with an alpha value of 0.4 to reduce noise in speed readings. The `calculateBearing` function computes the compass bearing from one coordinate to another using the arctangent of trigonometric components. The `filterFutureStops` function takes an array of stops and the current bus position and returns only those stops that are ahead of the bus based on cumulative distance. The `formatDistance` function converts meter values to human-readable strings such as 2.5 km or 350 m. The `calculateETA` function estimates arrival time in minutes by dividing distance by average speed.

6. Database Schema

The Firebase Realtime Database uses a JSON tree structure with multiple root-level nodes.

GPS Node

The `gps` node stores real-time bus telemetry data written by the ESP32 hardware. It contains three child fields. The `lat` field stores the current latitude as a double precision floating point number. The `lng` field stores the current longitude as a double precision floating point number. The `timestamp` field stores the time of the GPS reading as a formatted string in the DD slash MM slash YYYY HH colon MM colon SS format.

Passengers Node

The `passengers` node tracks bus occupancy. It contains two child fields. The `count` field stores the current physical passenger count as an integer, updated by the ESP32 IR sensor logic. The `booked` field stores the total number of tickets that have been booked for the current trip as an integer.

Routes Node

The `routes` node stores route definitions created by administrators. Each route is keyed by a unique `routeId` and contains multiple fields. The `id` field stores the route identifier string. The `name` field stores the display name of the route. The `bookingCount` field tracks the total number of bookings made for this route. The `totalDistance` field stores the route length in kilometers as a floating point number. The `timings` field contains a text description of the route duration.

The `stops` array within each route contains stop objects. Each stop object has an `order` field indicating its position in the sequence, a `name` field for display, a `lat` field for latitude, and a `lng` field for longitude.

The `schedules` array within each route contains schedule objects. Each schedule object has a `time` field in HH colon MM format and a `days` array containing integer indices for days of the week where zero represents Sunday and six represents Saturday.

The `userId` field associates the route with the administrator who created it.

Tickets Node

The tickets node stores student ticket records organized by `userId` and `ticketId`. Each ticket contains an `id` field for the ticket identifier, a `title` field describing the journey such as Main Campus to City Center, `from` and `to` fields for origin and destination stop names, a `date` field for the travel date, and a `routeId` field linking to the associated route.

Financial and tariff information includes a `price` field for the fare paid, a `tariff` field indicating the pass type such as 30min, 60min, or 24h, a `zones` field for zone information, and a `durationMs` field specifying ticket validity in milliseconds.

Status tracking includes a `status` field with values `active`, `expired`, or `used`, an `onboarded` boolean field indicating whether the ticket has been scanned by the driver, and a `createdAt` field with the ISO timestamp of creation.

The `seatReservations` array stores seat booking details. Each reservation contains a `seatNumber` integer, a `passengerName` string, an `age` integer, and a `gender` string.

Bookings Node

The bookings node stores seat reservations organized by `routeId` and seat number. Each booking record contains a `passengerName` string, an `age` integer, a `gender` string, a `bookedBy` field containing the user UID who made the booking, and a `timestamp` field with the ISO timestamp of the booking.

Users Node

The users node stores user profile information keyed by user UID. Each user record contains a `name` string, an `email` string, a `phone` string, a `role` string with values `student`, `admin`, or `driver`, a `balance` number for wallet funds, and a `createdAt` field with the ISO timestamp of account creation.

The `emergencyContact` field stores a phone number for emergency situations. The `addresses` array stores saved destination objects, each containing a `label` string such as `Home`, an `address` string, a `lat` double, and a `lng` double.

QR Logs Node

The `qr_logs` node maintains an audit trail of QR code scans. Each log entry contains a `ticketId` string identifying the scanned ticket, a `timestamp` string in DD slash MM slash YYYY HH colon MM colon SS format, a `status` string with values `success`, `error`, or `duplicate`, a `passengerCount` integer indicating the count after the scan, and a `ticketTitle` string.

Logs Node

The logs node records entry and exit events from both the ESP32 hardware and the driver application. Each log entry contains a `type` string with values `ENTRY`, `EXIT`, or `BOARDING`, a `passengerCount` integer, a `timestamp` string, an optional `passengerName` string, and an optional `ticketId` string.

SOS Node

The `sos` node stores emergency alert data. Each SOS record contains a `ticketId` string, a `passengerName` string, a `seatNumber` string, a `timestamp` string in ISO format, and a `status` string with values `active` or `resolved`.

Drivers Node

The drivers node stores presence markers for driver authentication. This node is used by Firebase security rules to verify write access permissions for driver operations.

7. Screens and Features

Student Screens

HomeScreen

The HomeScreen serves as the main dashboard for students. It displays a personalized greeting with the user name obtained from the Firebase users node. The screen uses Expo Location to obtain the current GPS position of the device. Live weather information is fetched from the OpenWeatherMap API based on the current coordinates.

The screen provides a stop picker interface with From and To fields that include search and swap functionality. A date picker allows selection of the travel date. When the user searches for routes, the screen displays a results list with bus cards showing departure time, price, and capacity information. Tapping a bus card expands the view to show a route detail visualization.

Active tickets are displayed as cards with live countdown timers showing remaining validity. Popular routes are presented as quick-select cards. The screen integrates several modal components including TopUpModal for wallet recharge, BusBookingModal for purchasing passes, ActiveTicketModal for viewing live ticket status, and SeatSelectionModal for choosing seats.

TicketsScreen

The TicketsScreen displays all tickets belonging to the authenticated user. Active tickets are shown in a separate section with live countdown timers that count down the remaining validity duration. Past and expired tickets are displayed in a history section below. Tapping on an active ticket opens the ActiveTicketModal which provides live tracking information and the QR code for boarding.

StopsScreen

The StopsScreen allows students to explore bus stops and view incoming buses. An autocomplete search field helps users find specific stops by name. The screen determines the nearest stop based on the device GPS location and highlights it. A horizontal scrollable tab bar allows quick switching between nearby stops.

For a selected stop, the screen lists incoming buses with estimated time of arrival calculated from current GPS positions and route data. A live Leaflet map embedded through WebView shows the current bus position on the route. Tapping on a bus opens a fullscreen status modal displaying telemetry statistics and a call-to-action for booking.

ProfileScreen

The ProfileScreen manages user account settings and personal information. It displays the user avatar, name, email address, and phone number retrieved from the Firebase users node. The wallet balance is shown with a button to open the TopUpModal for adding funds.

Users can manage saved destinations through a CRUD interface, adding frequently visited places with labels and GPS coordinates. An emergency contact number can be set and updated. An edit profile modal allows changing personal details. A logout button terminates the authenticated session.

Admin Screens

AdminDashboard

The AdminDashboard is a real-time command center for transit monitoring. It displays a Leaflet map through WebView showing the bus trail with a custom bus icon. Key performance indicator cards show current speed calculated from GPS data, passenger count from the passengers node, and capacity percentage.

An occupancy donut chart visualizes the ratio of filled seats to available seats. An hourly area chart shows passenger volume trends over time. Station predictions display boarding counts, alighting counts, and projected passenger numbers for each upcoming stop.

A real-time scrollable activity log shows entry and exit events as they occur. SOS emergency alerts appear as notifications when passengers trigger the emergency button. Toast notifications appear for boarding and alighting events. A fullscreen map mode expands the map view with overlay metrics.

RouteSetterScreen

The RouteSetterScreen provides route creation and management tools for administrators. It operates in two modes. List view shows all saved routes with edit and delete options. Builder mode provides an interactive route construction interface.

In builder mode, administrators can set a route name, add schedules using a time picker and day-of-week selectors, and search for stop locations using the Nominatim OpenStreetMap geocoding service. An interactive Leaflet map allows placing stop pins at precise locations. The stop sequence can be reordered through the interface. Route statistics including total distance and estimated duration are calculated and displayed automatically.

DriverModeScreen

The DriverModeScreen is the operator interface for ticket verification. It displays current passenger statistics including the number of passengers onboard, total booked tickets, and available seats remaining. Step-by-step scanning instructions guide the driver through the verification process.

A large Scan QR Ticket button opens the fullscreen QR scanner. The scanner uses Expo Camera with an animated scan line for visual feedback. When a QR code is detected, the system validates the ticket against the Firebase tickets node. It checks the onboarded flag to prevent duplicate scanning. The passenger count is incremented atomically using Firebase runTransaction to prevent race conditions.

On successful validation, a green animated modal displays the passenger information. On error, a red animated modal indicates the failure reason. All scans are logged to the qr_logs node for audit purposes.

Onboarding and Authentication Screens

WelcomeScreen

The WelcomeScreen is the initial screen shown to new users. It displays the application icon, the brand name Crowd Sense Hub in styled typography, and a tagline describing the application value proposition. A Get Started button transitions to the LoginScreen. The background uses a linear gradient.

LoginScreen

The LoginScreen provides authentication functionality with two modes. Login mode accepts email and password credentials for existing users. Sign Up mode creates new user accounts. The screen supports hardcoded admin credentials with the email admin at crowdsense dot com and password Admin2026 for administrator access.

During sign up, the application creates a Firebase Authentication user account and writes a corresponding profile to the Firebase users node with an initial wallet balance of 500. Form validation ensures required fields are completed correctly. A loading indicator appears during authentication operations.

8. Reusable Components

GlassCard

The GlassCard component is a wrapper component that provides a consistent card styling across the application. It implements a frosted-glass visual effect with a semi-transparent white background, rounded corners, drop shadows, and a subtle border. All child content passed to this component inherits the card styling.

QRScannerModal

The QRScannerModal is a fullscreen modal component for QR code scanning. It uses the Expo Camera library to render a camera preview with QR detection capabilities. When a QR code is scanned, the component validates the ticket against the Firebase tickets node. It checks whether the ticket has already been marked as onboarded to prevent duplicate scans. The passenger count is incremented atomically using Firebase runTransaction. All scan events are logged to the qr_logs node. Animated feedback displays green for successful scans and red for errors.

RouteVisualization

The RouteVisualization component renders an SVG-based route map. It displays only future stops by filtering out stops the bus has already passed. Stops are color-coded with green for the starting stop, violet for intermediate stops, and red for the terminal stop. A bus icon is positioned at the current GPS location. A dashed polyline connects the bus position to future stops. An information card shows the distance to the next stop and the estimated time of arrival.

TopUpModal

The TopUpModal provides wallet top-up functionality. Users can select quick amount options of 50, 100, 200, or 500 units, or enter a custom amount. When a top-up is confirmed, the component updates the user balance in the Firebase users node using the update operation.

BusBookingModal

The `BusBookingModal` manages ticket purchasing. It presents three pass options: a 30-minute pass for 30 units, a 60-minute pass for 50 units, and a 24-hour pass for 150 units. The component checks the user wallet balance before processing the purchase. On confirmation, it deducts the fare from the wallet and creates a new ticket in the Firebase tickets node under the user identifier.

BusDetail3DModal

The `BusDetail3DModal` displays comprehensive bus information. It renders an isometric three-dimensional SVG bus visualization. Live telemetry data including speed from the GPS node and crowd density from the passengers node is displayed. A route timeline shows the sequence of stops with estimated arrival times. A call-to-action button allows users to proceed to seat selection and booking.

ActiveTicketModal

The `ActiveTicketModal` is a draggable bottom-sheet modal that shows the active ticket status. It renders a QR code image generated through the qrserver API for driver scanning. A live Leaflet map displayed through WebView shows the current bus location. Real-time speed calculated using the Haversine formula is displayed. Passenger count and crowd density metrics are shown. A countdown timer indicates remaining ticket validity. An SOS button allows the passenger to trigger an emergency alert.

SeatSelectionModal

The `SeatSelectionModal` implements a full 70-seat bus layout selection interface. Seats are rendered in a grid layout with gender-based color coding where pink represents female passengers, dark represents male passengers, and amber represents the currently selected seat. The component enforces gender segregation rules for adjacent seats. A passenger details form collects name, age, and gender for each selected seat. Payment is processed from the wallet balance. On completion, bookings are written to the Firebase bookings node and the passenger count is updated.

9. Navigation and Routing

The application uses state-based conditional rendering for navigation rather than a declarative navigation library. The root `App.js` component manages a state machine that determines which screens to render based on authentication status and user role.

Navigation State Machine

When the application initializes, it loads fonts and checks the Firebase authentication state. If no user is authenticated, the application enters the unauthenticated state. In this state, the `WelcomeScreen` is displayed initially. After the user taps Get Started, the `showWelcome` state flag is set to false and the `LoginScreen` is rendered.

When a user successfully authenticates, the application checks the role field in the Firebase users node for that user. If the role is student, the application enters the student state and renders the student tab bar with Home, Tickets, Stops, and Profile screens. If the role is admin, the application enters the admin state and renders the admin tab bar with Dashboard, Routes, Driver Mode, and Logout screens.

Tab Bar Design

The student tab bar uses a custom floating pill-shaped design with animated icons. The admin tab bar uses a standard bottom bar design with icon and label pairs for each tab.

Internal Screen Transitions

Within the HomeScreen, three internal screen states are managed through a currentScreen state variable. The home state is the default and shows the search form and popular routes. The results state is triggered after a route search and displays bus card results. The detail state is triggered when a bus card is tapped and shows the route detail view with a booking button.

10. State Management and Data Flow

Data Flow Architecture

Data flows unidirectionally between the Firebase Realtime Database and the application state. Firebase real-time listeners push data changes to the application, which updates React component state and triggers UI re-renders. User actions trigger Firebase write operations, which propagate to all connected clients through the listener mechanism.

Real-Time Sync Strategy

The application uses three primary data synchronization mechanisms. Real-time listeners using the Firebase onValue method provide continuous synchronization for live data such as GPS coordinates, passenger counts, and SOS alerts. One-time reads using the Firebase get method fetch static data such as user profiles and route definitions that do not require continuous updates. Atomic writes using the Firebase runTransaction method ensure race-safe operations for passenger count increments during QR scanning.

Key State Variables

The root App.js manages three critical state variables: the user object from Firebase Authentication, the userRole string retrieved from the Firebase users node, and the fontsLoaded boolean from the font loading operation.

The HomeScreen manages location data from Expo Location, fromStop and toStop objects for the search form, the selected date, an array of route search results, and an array of active tickets.

The AdminDashboard manages GPS data from the gps node, passenger count from the passengers node, an array of log entries from the logs node, and an array of SOS alerts.

The RouteSetterScreen manages an array of stops for the route being built, an array of schedule objects, and the current map region.

The DriverModeScreen manages the current passenger count and the total booked count from the passengers node.

11. Authentication and Security

Authentication Flow

Authentication is handled through Firebase Authentication using email and password credentials. When a user enters their email and password, the application calls the Firebase `signInWithEmailAndPassword` function for existing users or the `createUserWithEmailAndPassword` function for new registrations. Firebase returns a JSON Web Token that confirms the authenticated session.

After authentication, the application reads the `uid` from the authentication result and queries the Firebase database at the path `users slash uid` to retrieve the user role. Based on the role value, the application renders the appropriate screen set.

Authentication State Persistence

The authentication state is persisted using `AsyncStorage` through the React Native Async Storage library. This allows the application to restore the authenticated session when the app is restarted without requiring the user to log in again.

Admin Authentication

Admin access uses a hardcoded credential check. When the email matches `admin at crowdsense dot com` with password `Admin2026`, the application grants administrator privileges. This provides a predefined administrative account for system management.

New User Registration

When a new user signs up, the application first creates a Firebase Authentication account using the provided email and password. It then writes a user profile to the Firebase database at the path `users slash uid` with the user name, email, phone number, role set to student, an initial wallet balance of 500, and a creation timestamp.

Security Rules

The Firebase Realtime Database uses security rules to enforce access control. Students can read and write only their own data under `users slash uid` and `tickets slash uid`. Drivers have write access to the `passengers`, `logs`, and `qr_logs` nodes. Administrators have full read and write access to all database nodes. Unauthenticated access is denied for all operations.

12. APIs and Third-Party Integrations

Firebase Authentication

Firebase Authentication provides the email and password authentication mechanism. The application uses the `signInWithEmailAndPassword` function for login and the `createUserWithEmailAndPassword` function for registration. The `onAuthStateChanged` listener monitors authentication state changes in real time.

Firebase Realtime Database

Firebase Realtime Database provides the primary data storage and synchronization layer. The application uses `ref` to create database references, `set` and `update` for writing data, `push` for generating unique child keys, `onValue` for real-time listeners, `get` for one-time reads, and `runTransaction` for atomic counter operations.

OpenWeatherMap

The OpenWeatherMap API provides live weather data for the student home screen. The application calls the current weather endpoint with the device latitude and longitude coordinates to retrieve temperature, weather condition, and icon information.

Nominatim OpenStreetMap

The Nominatim API provides geocoding services for the RouteSetterScreen. Administrators can search for stop locations by name, and the API returns matching coordinates and address information.

Leaflet JavaScript Library

Leaflet is an open-source JavaScript mapping library embedded through React Native WebView. It provides interactive map rendering with OpenStreetMap tile layers. The application uses custom divIcon markers for the bus position, circleMarker elements for stops, polyline elements for route paths, and CSS-based attribution hiding for a clean interface.

OpenStreetMap Tile Server

The OpenStreetMap tile server provides free map tile images for the Leaflet maps embedded in the application. All map visualizations including the admin dashboard, route builder, stops screen, and active ticket tracking use these tiles.

QR Server API

The QR Server API generates QR code images for active tickets. The application constructs a URL with ticket data and the API returns a PNG image that can be scanned by the driver application.

Nodemailer Gmail SMTP

The Express mailer server uses Nodemailer with Gmail SMTP credentials to send OTP verification emails. The server constructs HTML email messages with project branding and sends them through the Gmail email service.

13. IoT Hardware Integration

ESP32 Microcontroller

The ESP32 microcontroller by Espressif Systems is a 32-bit dual-core processor with integrated WiFi and Bluetooth capabilities. It serves as the physical data acquisition and transmission unit installed on the bus. The microcontroller runs firmware that reads sensor data, processes passenger detection logic, manages the display output, and communicates with Firebase over encrypted HTTPS connections.

GPS Module

The NEO-6M GPS module receives signals from GPS satellites and calculates the bus position using trilateration. The module outputs NMEA sentence strings over a serial UART connection at 9600 baud. The TinyGPSPlus library parses these sentences into usable latitude and longitude values. The ESP32 pushes the

latest valid GPS coordinates to the Firebase gps node every five seconds. If no valid GPS fix is available, coordinates of zero are pushed to indicate no signal.

IR Sensors for Passenger Detection

Two digital IR proximity sensors are mounted at the bus door in a sequential arrangement. The sensors use an infrared LED transmitter and photodiode receiver. When a person passes through the door, the sensors trigger in a specific sequence that determines the direction of movement.

If the outer sensor triggers first and the inner sensor triggers within three seconds, the system registers a passenger entry and increments the count. If the inner sensor triggers first and the outer sensor triggers within three seconds, the system registers a passenger exit and decrements the count. A three-second timeout prevents partial triggers from causing false detections. The count is clamped to a minimum of zero to prevent negative values from sensor glitches.

OLED Display

The SSD1306 OLED display with 128 by 64 pixel resolution connects to the ESP32 through the I2C protocol using custom pins. The display shows several screen states. During boot, a splash screen appears with a loading progress bar. During initialization, status messages indicate WiFi connectivity, Firebase authentication, and time synchronization progress. In normal operation, the idle screen shows the current passenger count in large text, the Indian Standard Time, and the GPS signal status. During passenger events, animated arrows and count change indicators provide visual feedback.

Audio Feedback

An active buzzer connected to a digital GPIO pin provides audio feedback for passenger events. A single beep indicates a passenger entry, and two beeps indicate a passenger exit. A single beep at startup confirms the system is ready.

Boot Sequence

When the ESP32 powers on, it initializes GPIO pins for the IR sensors and buzzer, starts the GPS UART at 9600 baud, initializes the I2C bus and OLED display, shows the splash screen, connects to WiFi, synchronizes time using NTP with the Indian Standard Time offset, authenticates with Firebase, restores the previous passenger count and log index from the Firebase database, sounds a ready beep, and enters the main processing loop.

Main Processing Loop

The main loop performs several tasks iteratively. It processes Firebase asynchronous tasks through the app dot loop function to maintain authentication tokens and dispatch pending operations. It feeds incoming GPS bytes to the TinyGPSPlus parser through the feedGPS function. If five seconds have elapsed since the last GPS push and Firebase is ready, it pushes the current GPS coordinates to the database. It manages OLED display animations using millis-based non-blocking timing. It reads the IR sensor states, detects rising edges, applies the direction detection logic, updates the passenger count and Firebase database when events are detected, and clears stale pending triggers after the three-second timeout.

14. Project Structure

Root Directory

The project root contains the application entry point file `index.js` that calls `registerRootComponent` with the `App` component. The `App.js` file is the root component handling authentication state management, font loading, and tab navigation. The `app.json` file contains Expo configuration including the application name `Crowd Sense Hub`, version `1.0.0`, portrait orientation, splash screen settings, and Android package identifier.

The `package.json` file lists all project dependencies and scripts. The `metro.config.js` file configures the Metro bundler with support for `mjs` file extensions. The `eas.json` file defines Expo Application Services build profiles for development, preview, and production environments. The `mailer-server.js` file contains the Express OTP mailer backend.

Source Directory

The `src` directory contains three subdirectories. The `config` subdirectory contains two files. The `firebase.js` file initializes the Firebase application with API credentials and exports the `app`, `database`, and authentication instances. The `gpsUtils.js` file exports pure utility functions for Haversine distance calculation, speed computation, bearing calculation, stop filtering, distance formatting, and ETA estimation.

The `components` subdirectory contains eight reusable component files. The `GlassCard.js` file implements the frosted-glass card wrapper. The `QRScannerModal.js` file implements the full-screen QR code scanner with Firebase validation. The `RouteVisualization.js` file implements the SVG route map with color-coded stops. The `TopUpModal.js` file implements the wallet top-up interface. The `BusBookingModal.js` file implements the pass purchase interface. The `BusDetail3DModal.js` file implements the bus information display with telemetry. The `ActiveTicketModal.js` file implements the live ticket tracking interface with QR code and map. The `SeatSelectionModal.js` file implements the 70-seat layout with gender segregation.

The `screens` subdirectory contains nine screen files. The `WelcomeScreen.js` file implements the onboarding splash screen. The `LoginScreen.js` file implements authentication with login and sign-up modes. The `HomeScreen.js` file implements the student main dashboard at approximately 863 lines of code. The `TicketsScreen.js` file implements the ticket list and history display. The `StopsScreen.js` file implements the stops exploration and live bus tracking at approximately 864 lines. The `ProfileScreen.js` file implements user profile management and settings at approximately 764 lines. The `AdminDashboard.js` file implements the real-time monitoring dashboard at approximately 1799 lines. The `RouteSetterScreen.js` file implements the route builder interface at approximately 2173 lines. The `DriverModeScreen.js` file implements the QR scanner panel at approximately 1082 lines.

Assets Directory

The `assets` directory contains image resources including the application icon, splash screen images, Android adaptive icon components, web favicon, and React logo images at multiple resolutions.

Source Code Metrics

The project source code spans approximately 12,400 lines across 23 source files. The `config` directory contains 2 files with approximately 120 lines. The `components` directory contains 8 files with approximately 1,800 lines. The `screens` directory contains 9 files with approximately 9,800 lines. The entry point and app files contain 3 files with approximately 600 lines. The backend mailer server contains 1 file with approximately 80 lines.

15. Conclusion

Key Achievements

The Bus Management System achieves a complete three-role ecosystem with tailored user interfaces for students, drivers, and administrators. Real-time synchronization through Firebase Realtime Database enables sub-second data propagation across all connected clients. IoT hardware integration allows the ESP32 microcontroller to seamlessly feed live sensor data into the cloud infrastructure.

Smart ticketing implementation provides QR-based boarding verification with duplicate prevention through onboarded flag checking and atomic counter operations using Firebase `runTransaction`. GPS bus tracking with Haversine-based distance calculations enables accurate ETA predictions and route visualization. The emergency system routes SOS alerts from passenger seats to the admin dashboard in real time.

Gender-segregated seat management implements a 70-seat layout with adjacency rules enforced during the booking process. The interactive route builder allows administrators to create and manage routes using map-based stop placement and schedule configuration. The admin dashboard provides comprehensive real-time monitoring with occupancy charts, activity logs, and live map tracking.

Intended Use Case

The system is designed for college and university campus bus fleets and small to medium transit operators requiring real-time visibility into bus occupancy, digital ticketing with QR verification, GPS-based fleet tracking, automated passenger counting through IoT sensors, and centralized administrative monitoring.

Future Roadmap

Planned enhancements include multi-bus support for tracking multiple vehicles simultaneously, offline mode with local storage buffering for connectivity loss scenarios, payment gateway integration for UPI and card payments, push notifications through Expo Notifications for boarding reminders and alerts, advanced analytics engine for ridership pattern analysis and predictive modeling, and multi-language support for regional language accessibility.

Documentation Version 1.0 — May 2026

Crowd Sense Hub — Bus Management System